

AI 100 講 — 通用導論

超級個體 全面向能力分佈

看懂四個面向

找到自己的能力分佈與學習路徑

按 → 或空白鍵開始

今天的旅程

- 1 時代命題：超級個體的崛起**
Vibe coding 讓單一強項不再是進場門檻，能力分佈才是
- 2 拆解四派各自的論點**
演算法派、軟體工程派、領域應用派、風險合規派 — 看穿銷售話術
- 3 解法：先站上交集點**
不是四派精通，是四派都「夠用就好」
- 4 站上之後：取得平衡**
依想成為什麼樣的超級個體，選方向深化
- 5 三份簡報的串接關係**
四面向 → L1-L4 階段 → 七層架構，從自己到細節

時代變了 — 單一強項不再是護城河

過去比技能深度，現在比能力分佈。

能落地的人，不是技能最深的人，是補位最全的人。



過去

單一深度技能 = 護城河
會寫程式、懂資料、懂行業
各自分工，互不重疊



現在

AI 把單一技能變廉價
不會寫的可以 vibe 出能跑的東西
不懂模型的也能用上模型



未來

能力分佈決定你能走多遠
覆蓋面廣的「超級個體」
一個人就是一支隊伍

💡 重點不是「我是不是這派的人」，是「我有沒有四派都夠用」。

但你會聽到很多聲音說...

「你不懂 ____,
vibe coding 就是在裸奔。」

每個派別都用這個句型，差別只在填什麼字

演算法派、軟體工程派、領域應用派、風險合規派
每派都告訴你「沒我這派你不行」。

🔑 接下來把這個套路拆開來看 — 每派強在哪、在賣什麼、看不到什麼。
看穿論述，你才不會被任一派綁架。

Part 2

拆解 四派的論點

每派強在哪、在賣什麼、看不到什麼

■ 演算法派 — 「不懂模型內部，你只是在亂試」



強項是真的

對「為什麼模型失敗」最有解釋力
能設計新模型
研究 / 高風險場景無可替代



在賣什麼

底層原理、微積分、論文導讀
「沒這個你只是工具人」
CS 學位 / ML 訓練營



看不到的事

CS 4 年訓練，70% 畢業後在呼叫 API
過度工程：很多場景用不到這深度
對 LLM 時代失準（傳統 ML $\neq$ prompt）

🔍 **怎麼辨識：**講課愛畫公式、推導、講論文，會說「不懂底層你就只是工具人」。

■ 軟體工程派 — 「沒架構紀律，過不了 production」



強項是真的

企業 production 場景無可替代
有完整工具鏈
design doc / CI-CD / 版控
對「為什麼長期會壞」解釋力最強



在賣什麼

工程紀律、TDD
specification engineering
「vibe coding 是垃圾」
「你寫的不是程式」
架構師訓練 / 開源社群名聲



看不到的事

把所有任務都當 production 做
GitHub stars 同溫層績效競賽
對非軟體背景的領域專家排斥

🔍 怎麼辨識：愛秀 GitHub repo、講架構圖、強調 best practices、批評「vibe coding 不認真」。

■ 領域應用派 — 「不懂行業，AI 都是空談」



強項是真的

速度極快、能直接交付
規模最大（4:1 超過專業開發者）
跟商業變現最直接



在賣什麼

「我用 AI 賺到多少」
行業 × AI 案例集
速度至上、變現速成



看不到的事

不知道自己不知道什麼（最危險）
3 個月後變自己也維護不了的黑盒
低估資料品質、安全、合規成本



怎麼辨識：講案例、講商業模式、強調速度與變現、「我用 AI 一週做了 X」。

■ 風險合規派 — 「不會判讀，你早晚會出事」



強項是真的

最接近跨領域共通能力
法規驅動（EU AI Act 強制要求）
對非技術背景者最友善



在賣什麼

AI 治理、合規培訓、ESG 顧問
「不是會用就好」
「責任歸屬不能跑」
用法規與恐懼拉抬重要性



看不到的事

容易變空泛說教（落地很弱）
跟具體技術 stack 銜接弱
圈內人有時自己不懂技術



怎麼辨識：講案例分析、講風險、強調法規與責任、「不是會用就好」。

真相 — 各派都有道理，也都有偏見



每派講的都不是錯的

強項都是真的、賣的東西都有價值、盲點也都真實存在



但每派都在保護自己領域的價值

同一個「你不懂 __，vibe coding 就是在裸奔」句型 = 同一個推銷模式



選邊站不是答案

選哪派都會錯過另外三派看到的事 — 而那剛好是你會踩坑的地方

你的任務不是選邊站，是看穿論述

找到自己的位置，補上看不到的地方

Part 3

先站上 交集點

不是四派精通，是四派都「夠用就好」

你心中真正的問題

「我對演算法不熟，
我就不能 vibe 出產品嗎？」

每個學員都有的疑惑

答案不是 yes / no，
真正的問題是：你的能力分佈長什麼樣？

💡 你不需要四派精通。但你需要四派都「夠用就好」 — 這就叫站上交集點。

解法 — 每派都到「夠用就好」

不需要四派精通（不可能），四派都「夠用就好」就辦得到。

夠用就好

■ 演算法派
不被 LLM 表象迷惑

夠用就好

■ 軟體工程派
知道何時要嚴肅工程

夠用就好

■ 領域應用派
能拆解真實需求

夠用就好

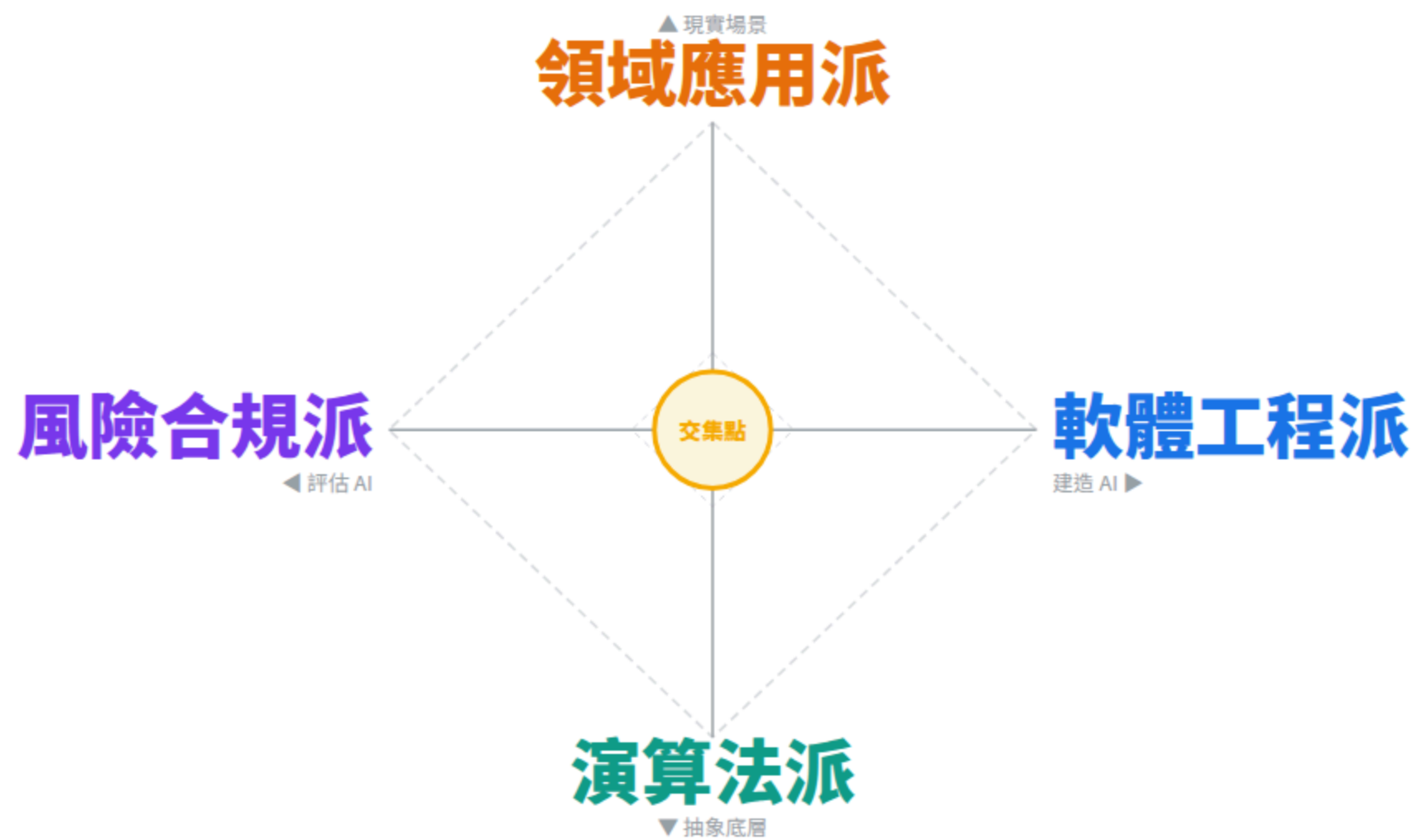
■ 風險合規派
會驗證、會劃紅線

站到交集點 = 你已經能 vibe 出能交付的產品

這時你是個能落地的**超級個體**，
不被任一派綁架，也不會在另外三派踩坑。

四面向能力分佈 — 雷達圖

縱軸：現實場景 ↔ 抽象底層 • 橫軸：評估 AI ↔ 建造 AI



中心 = 四派都「夠用就好」的位置

用蓋房子來想 — 四工種缺一不可

工地角色	對應的 AI 派別	負責的事
 結構技師	演算法派	算結構、算載重 — 看不見但決定整棟樓的安全
 建築師	軟體工程派	把需求變成可施工的設計圖
 工班師傅	領域應用派	水電、泥作各工種 — 把圖變成實體
 監造	風險合規派	對照規範，逐階段驗收

 四工種平行協作、缺一不可，但 沒有人是四工種同時頂尖 — 各自夠用、互補完成一棟房子。

AI 應用一樣 — 不需要四派精通，需要四派都能補位。

Part 4

站上之後 取得平衡

依想成為什麼樣的超級個體，選方向深化

四派「夠用就好」長什麼樣

每派列 3 條 — 不是教科書清單，是辨識題：你有沒有這個夠用？



演算法派

知道 LLM 是預測引擎不是真理引擎
知道幻覺常出現在引用 / 人名 / 數字
知道 cutoff 之後的東西要自己查



軟體工程派

知道 prototype 跟 production 怎麼分線
不 commit 自己看不懂的程式
API key 不寫在程式碼裡



領域應用派

能把模糊需求拆成輸入→處理→輸出→驗證
驗收標準事先定義，不是做完再看
MVP 先驗證再加功能



風險合規派

AI 給的引用 / 人名 / 數字會去查
客戶資料 / 個資不丟雲端 AI
知道什麼時候該說「這不該用 AI」

取得平衡 — 你想成為什麼樣的超級個體？

路徑	適合的人	策略
T 型 一深三淺	要做專業變現的人	站到交集點 + 把最強的一派深化到專家級
π 型 兩深兩淺	要做高價定位的人	同時深化兩派（如演算法 + 風險，或軟體 + 領域） ⚠ 成本是 T 型 2.5 倍，稀缺度 10 倍
滿格型 四派精通	不必走的路	不可能且沒必要 — 想做這個的人最後一派都不通

💡 四派都到「夠用」的人 = 0.01% ($0.1 \times 0.1 \times 0.1 \times 0.1$)。

乘法稀缺，不是加法 40%。

站上交集點本身就是稀缺定位。

你現在在哪？下一步補什麼？

你最常做的事	你大概偏哪派	下一步該補
讀論文、跑實驗、調模型	演算法派	軟體工程 + 領域應用 + 風險合規
寫 code、做架構、上 GitHub	軟體工程派	演算法 + 領域應用 + 風險合規
跟客戶聊、做提案、找需求	領域應用派	演算法 + 軟體工程 + 風險合規
寫評論、做培訓、講風險	風險合規派	演算法 + 軟體工程 + 領域應用

看完應該能說出三件事

1. 我偏 X 派
2. 我看不到 Y 派
3. 下一步先補 Y 派的「夠用就好」

Part 5

學習路徑 串接

四面向 → L1-L4 階段 → 七層架構，從自己到細節

三份是一條路 — 從自己看到細節

■ 四面向

本份 · 看自己

看自己的能力分佈 — 用四派找到自己偏哪派、看不到哪派、下一步補什麼。
回答的問題：「我現在在哪？」

■ L1-L4 階段

點擊開啟 →

看自己的學習路徑 — L1 用工具 → L2 選工具 → L3 創工具 → L4 流程改造，每階段對應不同的能力分佈。
回答的問題：「下一步該往哪走？」

■ 七層架構

點擊開啟 →

看 AI 技術全景 — L1 模型 → L7 應用，所有專有名詞都在這張圖找得到位置。
回答的問題：「這些工具、這些名詞，是在做什麼？」

💡 四面向看自己，L1-L4 看路徑，七層看細節 — 三份合起來才是完整的超級個體成長地圖。

今天的重點

1 時代要的是超級個體，不是單派專家

vibe coding 讓單一強項不再是進場門檻，能力分佈才是

2 四派各有論點，也各有偏見

每派都用「你不懂 __，vibe coding 就是在裸奔」這個句型推銷

3 解法不是選邊站，是先站上交集點

四派都「夠用就好」，這時你已經能 vibe 出產品

4 站上之後再選方向深化

四面向看自己 → L1-L4 看路徑 → 七層看細節

先站上交集點 再選方向深化

AI 100 講 — 通用導論

2026.04